

INSTITUTO POLITECNICO
NACIONAL
Escuela Superior de Computo

Arquitectura de las Computadoras

ALU

Profesor: Miguel Angel Aleman Arce

Alumno: Gutiérrez Espinosa Jordan Gabriel

Grupo: 5CV1

Ciclo: 2022-2023(1)

El objetivo del proyecto era diseñar una Unidad Aritmético-Lógica (ALU) utilizando el lenguaje de descripción de hardware Verilog. La ALU debía ser capaz de realizar seis operaciones en total, tres aritméticas (incluyendo la multiplicación) y tres lógicas. También se requería la implementación de una operación de RESET y una operación NOP.

El diseño de la ALU se basa en un módulo llamado "ALU". El módulo tiene varias entradas y salidas, incluyendo una señal de reloj (*clk*), una señal de reinicio (*reset*), una señal de inicio (*start*) para dar inicio a las operaciones, dos operandos de 8 bits (*operand1* y *operand2*), un código de operación de 3 bits (*opcode*), un resultado de 16 bits (*result*), y una señal de finalización (*done*) como se muestra en la Figura 1.

```
module ALU (  
    input wire clk,  
    input wire reset,  
    input wire start,  
    input wire [7:0] operand1,  
    input wire [7:0] operand2,  
    input wire [2:0] opcode,  
    output reg [15:0] result,  
    output wire done  
);  
  
    reg [7:0] reg_operand1;  
    reg [7:0] reg_operand2;  
    reg [2:0] reg_opcode;  
    reg [15:0] reg_result;  
    reg reg_done;
```

Figura 1, Inicio de las variables.

El código Verilog implementa la funcionalidad de la ALU utilizando una máquina de estado secuencial sincronizada por flanco de subida del reloj. Cuando se recibe una señal de reinicio (*reset*), se restablecen las señales *done* y *result*.

Cuando se recibe una señal de inicio (*start*), se carga el primer operando (*operand1*), el segundo operando (*operand2*) y el código de operación (*opcode*) en registros internos. Luego, se utiliza una instrucción case para seleccionar la operación correspondiente según el código de operación y se realiza el cálculo correspondiente. El resultado se almacena en un registro interno (*reg_result*), y la señal *done* se establece en 1 para indicar que la operación ha finalizado, Como se muestra en la Figura 2.

```

always @(posedge clk) begin
  if (reset) begin
    reg_done <= 0;
    reg_result <= 0;
  end else begin
    if (start) begin
      reg_done <= 0;
      reg_operand1 <= operand1;
      reg_operand2 <= operand2;
      reg_opcode <= opcode;
      case (reg_opcode)
        3'b000: reg_result <= reg_operand1 + reg_operand2; // Suma
        3'b001: reg_result <= reg_operand1 - reg_operand2; // Resta
        3'b010: reg_result <= reg_operand1 * reg_operand2; // Multiplicación
        3'b011: reg_result <= reg_operand1 & reg_operand2; // AND lógico
        3'b100: reg_result <= reg_operand1 | reg_operand2; // OR lógico
        3'b101: reg_result <= reg_operand1 ^ reg_operand2; // XOR lógico
        default: reg_result <= 16'd0; // NOP (no operation)
      endcase
      reg_done <= 1;
    end else begin
      reg_done <= 0;
    end
  end
end

```

Figura 2, Operadores y selección

Se ha agregado un bloque *always* adicional para asignar correctamente el valor del registro interno *reg_result* a la salida *result*.

```

assign done = reg_done;

always @(reg_result) begin
  result <= reg_result;
end

ndmodule

```

Figura 3, Resultados.

el código proporcionado implementa con éxito una ALU en Verilog que puede realizar seis operaciones diferentes se puede representar en compuestas lógicas como se muestra en la Figura 4. Proporciona la flexibilidad para agregar más funcionalidades según sea necesario y puede ser utilizado como punto de partida para proyectos más complejos relacionados con el diseño de hardware digital, ya por último se muestra su ejecución en la Figura 5.

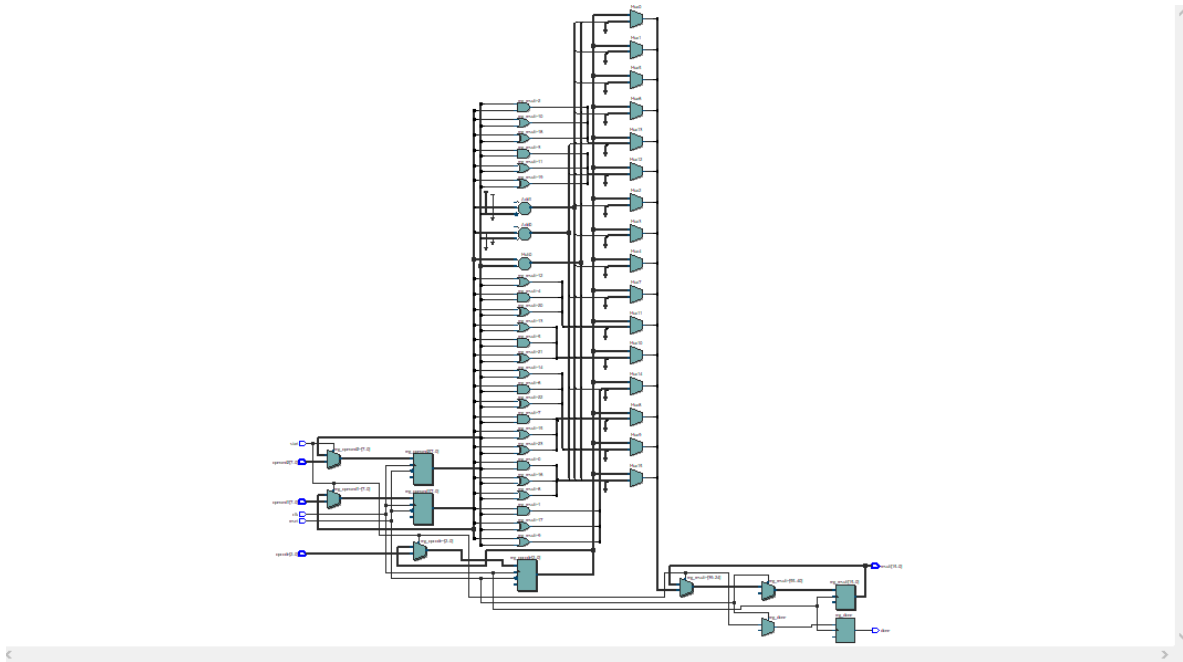


Figura 4, ALU con compuestas lógicas.

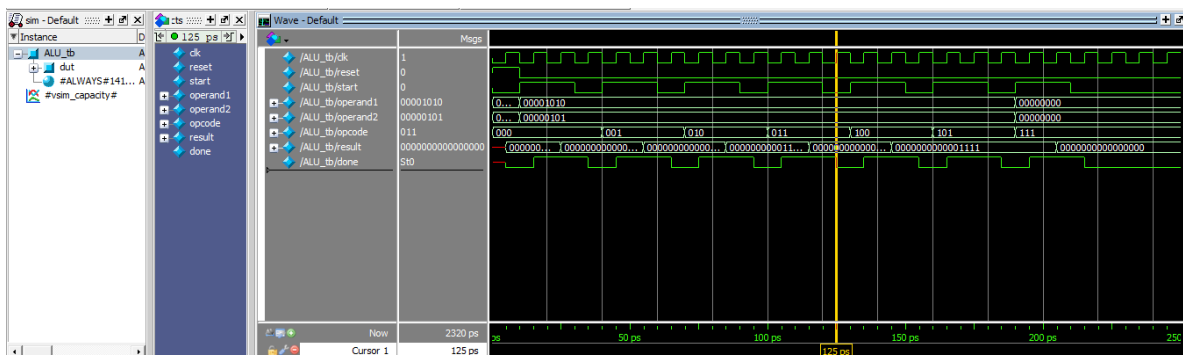


Figura 5, Simulación.